Detailed Line-by-Line Explanation of ExcelBulkUpload.js

File Reference:

This component is a React frontend module used for:

- Uploading Excel files (.xlsx/.xls)
- Validating Excel columns
- Converting Excel data into JSON
- Sending bulk data to backend API
- Displaying success/error messages
- Downloading sample Excel format
- Preventing duplicate upload after success

---

1. IMPORT SECTION

```
import React, { useState } from "react";
```

Explanation

- React → Required to create React components.
- useState → React Hook used to store and manage component state.

```
import {
  Paper,
  Typography,
  Button,
  Box,
  Alert,
  CircularProgress,
} from "@mui/material";
```

Explanation

These are Material UI components used for UI design.

Component	Purpose
Paper	Card-like container
Typography	Text headings and labels
Button	Clickable button
Box	Layout wrapper
Alert	Success/Error message
CircularProgress	Loading spinner

```
import * as XLSX from "xlsx";
```

Explanation

Imports the xlsx library.

Used for:

- Reading Excel files
- Converting sheets to JSON

---

import axios from "axios";

Explanation

axios is used for API calls to backend server.

Example:

axios.post(...)

---

import CloudQueueRoundedIcon from "@mui/icons-material/CloudQueueRounded";

Explanation

Imports upload cloud icon from Material UI Icons.

---

2. COMPONENT DEFINITION

const ExcelBulkUpload = (props) => {

Explanation

Creates a functional React component named:

ExcelBulkUpload

---

3. PROPS EXTRACTION

const { user, serverIp } = props;

Explanation

Extracts values passed from parent component.

Variable	Purpose
user	Logged-in user details
serverIp	Backend API base URL

---

4. STATE VARIABLES

const [error, setError] = useState("");

Explanation

Stores error messages.

Example:

"Invalid Excel format"

---

const [success, setSuccess] = useState("");

Explanation

Stores success messages.

---

const [loading, setLoading] = useState(false);

Explanation

Controls loading spinner while uploading data.

---

const [excelData, setExcelData] = useState([]);

Explanation

Stores converted Excel rows in array format.

Example:

```
[
 { tenderName: "ABC", customerName: "XYZ" }
]
```

---

const [isUploaded, setIsUploaded] = useState(false);

Explanation

Used to:

- Hide upload button after successful upload
- Prevent duplicate uploads

---

5. SAMPLE FILE DOWNLOAD FUNCTION

const handleDownloadSampleExcel = () => {

Explanation

Function triggered when user clicks:

Sample format

---

const fileUrl = "/sample/Domestic_Lead_Sample.xlsx";

Explanation

Path of sample Excel file stored in frontend public folder.

---

const link = document.createElement("a");

Explanation

Creates HTML anchor tag dynamically.

---

link.href = fileUrl;

Explanation

Sets file path for download.

---

link.download = "Domestic_Lead_Sample.xlsx";

Explanation

Specifies filename during download.

---

document.body.appendChild(link);

Explanation

Adds link temporarily to DOM.

---

link.click();

Explanation

Automatically triggers file download.

---

document.body.removeChild(link);

Explanation

Removes temporary link after download.

---

6. DATABASE COLUMN ARRAY

const DB_COLUMNS = [

Explanation

Defines all database fields required for upload.

These are:

- Excel fields
- User metadata fields

Example

"tenderName"

Means:

Excel column must contain:

tenderName

Important User Fields

"OperatorId",

"OperatorName",

"OperatorRole",

"OperatorSBU",

These are automatically added from logged-in user.

Not taken from Excel.

---

7. COLUMN MATCH ARRAY

const DB_COLUMNS_MATCH = [

Explanation

Used ONLY for Excel validation.

This checks:

- Missing columns
- Extra columns

Difference Between Two Arrays

Array	Purpose
DB_COLUMNS	Final object creation
DB_COLUMNS_MATCH	Excel validation

---

8. HANDLE FILE UPLOAD FUNCTION

const handleFileUpload = (event) => {

Explanation

Runs when user selects Excel file.

---

const file = event.target.files[0];

Explanation

Gets uploaded file.

---

if (!file) return;

Explanation

Stops function if no file selected.

---

setError("");

setSuccess("");

Explanation

Clears previous messages.

---

9. FILE READER

const reader = new FileReader();

Explanation

Reads uploaded Excel file.

---

reader.onload = (e) => {

Explanation

Runs after file is completely loaded.

---

10. CONVERT FILE TO BINARY

const data = new Uint8Array(e.target.result);

Explanation

Converts file into binary array format.

Required by XLSX library.

---

11. READ EXCEL WORKBOOK

const workbook = XLSX.read(data, { type: "array" });

Explanation

Reads Excel workbook.
Workbook = entire Excel file.

---

12. GET FIRST SHEET

```
const sheetName = workbook.SheetNames[0];
```

Explanation

Gets first sheet name.

```
const worksheet = workbook.Sheets[sheetName];
```

Explanation

Gets sheet object.

---

13. CONVERT SHEET TO JSON

```
const jsonData = XLSX.utils.sheet_to_json(worksheet, { header: 1 });
```

Explanation

Converts Excel rows into array format.

Example:

```
[
 ["name", "age"],
 ["John", 22]
]
```

---

14. EMPTY FILE CHECK

```
if (jsonData.length === 0)
```

Explanation

Checks if Excel is empty.

---

15. EXTRACT COLUMN NAMES

```
const excelColumns = jsonData[0].map((col) => col.toString().trim());
```

Explanation

Gets first row (header row).

Removes extra spaces.

---

16. VALIDATE MISSING COLUMNS

```
const missing = DB_COLUMNS_MATCH.filter(
  (col) => !excelColumns.includes(col)
);
```

Explanation

Checks required columns missing in Excel.

---

17. VALIDATE EXTRA COLUMNS

```
const extra = excelColumns.filter(
  (col) => !DB_COLUMNS_MATCH.includes(col)
);
```

Explanation

Checks unwanted extra columns.

---

18. COLUMN MISMATCH ERROR

```
if (missing.length > 0 || extra.length > 0)
```

Explanation

If:

- Missing columns exist
  OR
- Extra columns exist

Show error.

---

19. CONVERT ROWS INTO OBJECTS

```
const rows = jsonData.slice(1).map((row) => {
```

Explanation

- Removes header row
- Processes actual data rows

```
const rowObj = {};
```

Explanation

Creates empty object for each row.

---

20. LOOP THROUGH DATABASE COLUMNS

```
DB_COLUMNS.forEach((col, index) => {
```

Explanation

Maps Excel columns into object properties.

---

21. AUTO ADD USER DETAILS

```
if (col === "OperatorId")
```

Explanation

Automatically inserts logged-in user ID.

```
rowObj[col] = user?.id || "291536";
```

Explanation

If user exists:

- use logged-in user ID

Otherwise:

- fallback default ID

Similar Blocks

OperatorName

OperatorRole

OperatorSBU

All insert logged-in user information automatically.

---

22. NORMAL EXCEL VALUE MAPPING

rowObj[col] = row[index] || "";

Explanation

Maps Excel column value into object.

Example:

```
{
  tenderName: "ABC"
}
```

---

23. STORE FINAL DATA

setExcelData(rows);

Explanation

Stores all processed rows into state.

---

setIsUploaded(false);

Explanation

Shows upload button again for new upload.

---

setSuccess(` File validated successfully. Total rows: ${rows.length}`);

Explanation

Shows success message.

---

24. READ FILE AS BUFFER

reader.readAsArrayBuffer(file);

Explanation

Starts reading Excel file.

---

25. UPLOAD TO BACKEND FUNCTION

const handleUploadToServer = async () => {

Explanation

Uploads validated Excel data to backend API.

---

26. EMPTY DATA CHECK

if (excelData.length === 0)

Explanation
Stops upload if no data exists.

---

27. REQUIRED FIELD VALIDATION
const invalidRows = excelData.filter((row) => !row.segment || !row.project);
Explanation
Checks:

- segment missing
  OR
- project missing

---

28. SHOW INVALID ROW ERROR
if (invalidRows.length > 0)
Explanation
Stops upload if required fields missing.

---

29. LOADING START
setLoading(true);
Explanation
Shows loading spinner.

---

30. API CALL
const response = await axios.post(
Explanation
Sends data to backend.

API URL
`${serverIp}/domesticLeadsBulkUpload`
Explanation
Backend endpoint.

REQUEST BODY

```
{
  excelData: excelData,
  OperatorId: user?.id || "291536",
  OperatorName: user?.username || "Default User",
}
```

Explanation
Backend receives:

- all Excel rows
- uploader information

---

31. SUCCESS RESPONSE

if (response?.data?.success)

Explanation

Checks backend upload success.

---

setSuccess("✅ Data successfully pushed into the database!");

Explanation

Displays success alert.

---

setIsUploaded(true);

Explanation

Hides upload button after success.

Prevents duplicate insertion.

---

32. ERROR HANDLING

catch (error)

Explanation

Handles:

- API failure
- server error
- network error

---

error?.response?.data?.message

Explanation

Gets backend error message safely.

---

33. FINALLY BLOCK

```
finally {
  setLoading(false);
}
```

Explanation

Stops loading spinner whether success or error.

---

34. HANDLE BULK UPLOAD FUNCTION

const handleBulkUpload = async () => {

Explanation

Another upload function.

But currently:

- NOT used in UI
- duplicate logic

You can remove this function safely if unused.

---

35. RETURN UI SECTION

return (

Explanation

Starts frontend UI rendering.

---

36. MAIN CONTAINER

<Box

Explanation

Main page wrapper.

Contains:

- gradient background
- center alignment
- spacing

---

37. PAPER COMPONENT

<Paper

Explanation

Creates card container.

---

38. SAMPLE FORMAT BUTTON

```
<Button
  onClick={handleDownloadSampleExcel}
>
```

Explanation

Downloads sample Excel template.

---

39. TITLE

<Typography variant="h5">

Explanation

Displays heading:

Upload Lead Enquiry Data

---

40. CONDITIONAL FILE UPLOAD BOX

{excelData.length === 0 && (

Explanation

Shows upload box ONLY before file selection.

---

41. BROWSE FILE BUTTON

<input

type="file"
accept=".xlsx,.xls"

Explanation

Allows only Excel files.

---

42. HANDLE FILE CHANGE

onChange={handleFileUpload}

Explanation

Triggers Excel reading when file selected.

---

43. ERROR ALERT

{error && (
<Alert severity="error">

Explanation

Shows error message if exists.

---

44. SUCCESS ALERT

{success && (
<Alert severity="success">

Explanation

Shows success message.

---

45. PUSH BUTTON

{excelData.length > 0 && !isUploaded && (

Explanation

Shows upload button only:

- after valid Excel upload
- before successful DB upload

---

46. PUSH BUTTON ACTION

onClick={handleUploadToServer}

Explanation

Uploads data to backend.

---

47. DISABLE BUTTON DURING LOADING

disabled={loading}

Explanation

Prevents multiple clicks while uploading.

---

48. LOADING SPINNER

{loading ? (

```
 <CircularProgress />
) : (
 "Push Data to Database"
)}
```

Explanation

Shows:

- Spinner while uploading
- Button text normally

---

49. EXPORT COMPONENT

```
export default ExcelBulkUpload;
```

Explanation

Makes component available for import in other files.

Example:

```
import ExcelBulkUpload from "./ExcelBulkUpload";
```

COMPLETE FLOW OF THIS COMPONENT

User uploads Excel
↓
Excel read using FileReader
↓
Converted to JSON using XLSX
↓
Columns validated
↓
Rows converted into objects
↓
Stored in excelData state
↓
User clicks Push Data
↓
Validation for required fields
↓
Axios API call to backend
↓
Data inserted into DB
↓
Success/Error shown

Detailed Line-by-Line Explanation of ViewLeadEnquiry.js

File Reference:

This React component is a complete **Lead Enquiry Management System UI** used for:

- Viewing lead enquiries
- Searching/filtering records
- Sorting records
- Editing records
- Deleting records
- Exporting Excel reports
- Dynamic column visibility
- Viewing detailed lead info inside dialogs

---

1. IMPORT SECTION

```
import React, { useEffect, useState } from "react";
```

Explanation

Import	Purpose
React	Required to create React component
useState	Used to store/manage state
useEffect	Used to run API calls after component loads

---

2. MATERIAL UI COMPONENT IMPORTS

```
import {
  Paper,
  Typography,
  TextField,
  Button,
  MenuItem,
  Box,
  TableContainer,
  TableHead,
  TableRow,
  TableCell,
  TableBody,
  Table,
  IconButton,
  Tooltip,
  Chip,
  Stack,
  Dialog,
  DialogTitle,
```

```
  DialogContent,
  DialogActions,
  InputAdornment,
  Checkbox,
  Menu,
} from "@mui/material";
```

Explanation

These are Material UI components used for frontend design.

Important Components

Component	Purpose
Table	Displays lead data
Dialog	Popup modal
TextField	Input fields
Button	Buttons
Tooltip	Hover tooltip
Checkbox	Column visibility selection
Menu	Dropdown menu
Chip	Small label/badge

---

3. ICON IMPORTS

```
import {
  SearchRounded,
  NorthRounded,
  SouthRounded,
  RestartAltRounded,
  EditRounded,
  DeleteRounded,
  CloseRounded,
} from "@mui/icons-material";
```

Explanation

Material UI icons.

Icons Used

Icon	Purpose
SearchRounded	Search icon
NorthRounded	Sort ascending
SouthRounded	Sort descending

Icon	Purpose
EditRounded	Edit action
DeleteRounded	Delete action
CloseRounded	Close dialog

---

import ViewColumnIcon from "@mui/icons-material/ViewColumn";

Explanation

Used for:

- Show/Hide column selection menu

---

4. XLSX IMPORT

import * as XLSX from "xlsx";

Explanation

Used for:

- Excel export functionality

---

5. AXIOS IMPORT

import axios from "axios";

Explanation

Used for:

- API calls
- Backend communication

---

6. TABLE HEADER STYLE

const headerCellStyle = {

Explanation

Reusable style object for table headers.

Example

fontWeight: 800

Makes header bold.

---

background: "linear-gradient(90deg, #001F54, #034078)",

Creates blue gradient header background.

---

7. DROPDOWN LISTS

const DROP_DOWN_LISTS = {

Explanation

Stores dropdown options.

Used in:

- Filters
- Edit forms

Example
inputType: ["EOI", "RFI", "RFE"]
These become selectable dropdown values.

---

8. TODAY DATE
const todayDate = new Date().toLocaleDateString("en-IN");
Explanation
Gets today's date in Indian format.
Example:
29/05/2026
Used in exported Excel filename.

---

9. COMPONENT DEFINITION
const ViewLeadEnquiry = (props) => {
Explanation
Main React component.

---

10. EXTRACT PROPS
const { serverIp, submitSuccess } = props;
Explanation
Gets values from parent component.

Prop	Purpose
serverIp	Backend API URL
submitSuccess	Detect new submissions

---

11. MAIN TABLE STATE
const [tableData, setTableData] = useState([]);
Explanation
Stores all lead enquiry rows.

---

12. SEARCH STATES
const [searchTerm, setSearchTerm] = useState("");
Explanation
Stores search box text.

---

const [inputTypeFilter, setinputTypeFilter] = useState("all");
Explanation

Stores selected input type filter.

---

13. SORT STATES

const [sortBy, setSortBy] = useState("dateCreated");

Explanation

Stores sorting field.

---

const [sortDirection, setSortDirection] = useState("desc");

Explanation

Stores:

- ascending
- descending

---

14. DIALOG STATES

const [selectedRow, setSelectedRow] = useState(null);

Explanation

Stores currently selected row.

---

const [editDialogOpen, setEditDialogOpen] = useState(false);

Explanation

Controls edit dialog visibility.

---

const [editingRow, setEditingRow] = useState(null);

Explanation

Stores currently editing row data.

---

15. DELETE DIALOG STATES

const [confirmDeleteOpen, setConfirmDeleteOpen] = useState(false);

Explanation

Controls delete confirmation popup.

---

const [idDeleteOpen, setIdDeleteOpen] = useState(null);

Explanation

Stores ID of record being deleted.

---

16. EDIT MODE STATES

const [isEditMode, setIsEditMode] = useState(false);

Explanation

Controls:

- view mode
- edit mode

---

17. COLUMN MENU STATES

const [columnMenuAnchor, setColumnMenuAnchor] = useState(null);

Explanation

Stores column menu position.

---

const columnMenuOpen = Boolean(columnMenuAnchor);

Explanation

Checks if column menu is open.

---

18. API URL

const LEAD_ENQUIRY_API = "/lead-enquiry";

Explanation

Backend API endpoint.

---

19. FETCH DATA USING useEffect

useEffect(() => {

Explanation

Runs automatically:

- on page load
- when submitSuccess changes

---

axios.get(serverIp + LEAD_ENQUIRY_API)

Explanation

Fetches lead data from backend.

---

setTableData(response.data.data);

Explanation

Stores API response in tableData state.

---

20. TOTAL ROW COUNT

const totalRows = tableData?.length || 0;

Explanation

Counts total records safely.

---

21. COLUMN DEFINITIONS

const leadColumns = [

Explanation

Defines all table columns.

Example

{ id: "leadReferenceNo", label: "Lead Ref No" }

Property	Meaning
id	Database field
label	Table header

---

22. COLUMN VISIBILITY STATE

const [visibleColumns, setVisibleColumns] = useState(

Explanation

Controls which columns are visible.

---

leadColumns.reduce((acc, col) => {

Explanation

Creates object like:

```
{
 leadName: true,
 customerName: true
}
```

All columns visible initially.

---

23. FILTER + SORT LOGIC

const filteredSortedData =

Explanation

Creates:

- filtered data
- sorted data

---

24. SEARCH FILTER

const q = searchTerm.toLowerCase();

Explanation

Converts search text to lowercase.

---

row.leadName?.toLowerCase().includes(q)

Explanation

Searches inside:

- leadName
- customerName
- leadOwner
- customerAddress

---

25. INPUT TYPE FILTER

inputTypeFilter === "all"

Explanation

If "all":

- show everything

Else:

- filter by selected input type

---

26. SORTING LOGIC

.sort((a, b) => {

Explanation

Sorts records.

---

if (aVal < bVal)

Explanation

Compares values.

---

return sortDirection === "asc" ? -1 : 1;

Explanation

Controls ascending/descending.

---

27. EXPORT FILTERED DATA

const filteredBQLeads = (filteredSortedData) => {

Explanation

Creates export data with only visible columns.

---

28. EXPORT TO EXCEL FUNCTION

const exportToExcel = (data, fileName = "export.xlsx") => {

Explanation

Exports table data into Excel.

---

29. CREATE WORKSHEET

const worksheet = XLSX.utils.json_to_sheet(data);

Explanation

Converts JSON into Excel sheet.

---

30. CREATE WORKBOOK

const workbook = XLSX.utils.book_new();

Explanation

Creates new Excel workbook.

---

31. APPEND SHEET

XLSX.utils.book_append_sheet(workbook, worksheet, "Sheet1");

Explanation

Adds worksheet into workbook.

---

32. GENERATE BASE64

const excelBase64 = XLSX.write(workbook, {

Explanation

Creates downloadable Excel binary.

---

33. CREATE BLOB

const blob = new Blob([byteArray], {

Explanation

Creates downloadable file object.

---

34. TRIGGER DOWNLOAD

link.click();

Explanation

Starts Excel download automatically.

---

35. SEARCH HANDLER

const handleSearchChange = (e)

Explanation

Updates search text state.

---

36. COLUMN MENU HANDLERS

const handleColumnMenuOpen = (event)

Explanation

Opens column visibility menu.

---

setColumnMenuAnchor(event.currentTarget);

Stores button location.

---

37. COLUMN TOGGLE

const handleColumnToggle = (columnId)

Explanation

Show/hide selected column.

---

38. SORT DIRECTION TOGGLE

const toggleSortDirection = ()

Explanation

Switches:

- asc
- desc

---

39. RESET FILTERS

const handleResetFilters = ()

Explanation

Resets:

- search
- filters
- sorting

---

40. ROW CLICK HANDLER

const handleRowClick = (row)

Explanation

Opens dialog in:

- view mode

---

41. EDIT CLICK HANDLER

const handleEditClick = (row)

Explanation

Opens dialog prepared for editing.

---

42. DELETE CLICK

const handleDeleteClick = (id)

Explanation

Opens delete confirmation popup.

---

43. EDIT FIELD CHANGE

const handleEditFieldChange = (field, value)

Explanation

Updates edited field dynamically.

---

44. ENTER EDIT MODE

setIsEditMode(true);

Explanation

Switches dialog into editable form mode.

---

45. SAVE CHANGES

const handleEditSave = ()

Explanation

Opens save confirmation popup.

---

46. CONFIRM SAVE

```
const handleConfirmSave = async ()
```

Explanation

Actually updates backend database.

---

47. UPDATE PAYLOAD

```
const updatePayload = {
  id: editingRow.id,
  ...editingRow,
};
```

Explanation

Creates updated object for backend.

---

48. UPDATE API CALL

```
await axios.put(
  `${serverIp}/lead-enquiry`,
  updatePayload
);
```

Explanation

Updates lead in database.

---

49. UPDATE LOCAL TABLE

```
const updatedTableData = tableData.map((row) =>
```

Explanation

Updates frontend table instantly without reload.

---

50. DELETE API

```
await axios.delete(`${serverIp}/getDomesticLead`
```

Explanation

Deletes record from backend.

---

51. REMOVE FROM UI

```
setTableData(
  tableData.filter((item) => item.id !== id)
);
```

Explanation

Removes deleted row from frontend table.

---

52. MAIN RETURN UI

```
return (
```

Explanation
Starts rendering UI.

---

53. HEADER SECTION
<Typography variant="h5">
Explanation
Displays page title:
Lead Data View

---

54. TOTAL ENTRIES CHIP
<Chip
label={`Total Entries: ${totalRows}`}
Explanation
Displays total row count.

---

55. SEARCH FIELD
<TextField
placeholder="Search title..."
Explanation
Search input box.

---

56. COLUMN TOGGLE BUTTON
<IconButton onClick={handleColumnMenuOpen}>
Explanation
Opens column visibility menu.

---

57. COLUMN MENU
<Menu>
Explanation
Dropdown menu for:

- show/hide columns

---

58. FILTER DROPDOWN
<TextField select>
Explanation
Dropdown filter for Input Type.

---

59. SORT BUTTON
<IconButton onClick={toggleSortDirection}>
Explanation
Changes sort order.

---

60. RESET BUTTON

<Button onClick={handleResetFilters}>

Explanation

Clears all filters.

---

61. DOWNLOAD REPORT BUTTON

<Button

onClick={() => {

Explanation

Exports filtered table into Excel.

---

62. TABLE CONTAINER

<TableContainer component={Paper}>

Explanation

Scrollable table wrapper.

---

63. TABLE HEADERS

<TableHead>

Explanation

Creates table header row.

---

64. ACTION COLUMN

<TableCell>

Actions

</TableCell>

Explanation

Contains:

- Edit button
- Delete button

---

65. SERIAL NUMBER

{index + 1}

Explanation

Displays row numbering.

---

66. TABLE ROW CLICK

onClick={() => handleRowClick(row)}

Explanation

Opens detail dialog.

---

67. EDIT BUTTON

<EditRounded />

Explanation

Opens edit mode dialog.

---

68. DELETE BUTTON

<DeleteRounded />

Explanation

Opens delete confirmation.

---

69. DYNAMIC COLUMN DISPLAY

leadColumns.map((col)

Explanation

Creates table dynamically based on visible columns.

---

70. DIALOG COMPONENT

<Dialog

open={editDialogOpen}

Explanation

Popup window for:

- viewing
- editing

---

71. DIALOG HEADER

Lead Enquiry Details

Explanation

Shows lead reference number.

---

72. MODE CHIP

isEditMode ? "EDIT MODE" : "VIEW MODE"

Explanation

Displays current dialog mode.

---

73. CUSTOMER INFORMATION SECTION

Customer Information

Explanation

Displays customer details.

---

74. CONDITIONAL EDITING

{isEditMode ? (

Explanation

If edit mode:

- show TextField

Else:

- show Typography

---

75. SAVE BUTTON

<Button onClick={handleEditSave}>

Explanation

Triggers save confirmation.

---

76. CONFIRMATION DIALOG

<Dialog open={confirmSaveOpen}>

Explanation

Confirmation popup before saving.

---

77. DELETE CONFIRMATION

<Dialog open={confirmDeleteOpen}>

Explanation

Confirmation popup before delete.

---

78. EXPORT COMPONENT

export default ViewLeadEnquiry;

Explanation

Exports component for usage elsewhere.

COMPLETE FLOW OF THIS COMPONENT

Load Component

↓

Fetch API Data

↓

Store in tableData

↓

Display in Dynamic Table

↓

Search / Filter / Sort

↓

Open Dialog on Row Click

↓

Edit/Delete Actions

↓

Send API Request

↓

Update Table UI

↓

Export Excel Report

Detailed Line-by-Line Explanation of FormLeadEnquiry.js

File Reference:

This file is the **main form component** used for:

- Creating Lead Enquiry records
- Handling form validations
- Submitting data to backend API
- Showing success message
- Resetting form
- Managing dropdowns dynamically

---

1. Import Statements

```
import { useState } from "react";
```

Explanation

Imports React Hook useState.

Used for:

- storing submitted data
- managing dropdown states

```
import { useForm, Controller } from "react-hook-form";
```

Explanation

Imports utilities from React Hook Form library.

Import	Purpose
useForm	Creates form handling system
Controller	Connects MUI inputs with React Hook Form

Material UI Imports

```
import {
  Container,
  Paper,
  Typography,
  TextField,
  Button,
  Grid,
  MenuItem,
  Box,
  Divider,
  Card,
  InputAdornment,
} from "@mui/material";
```

Explanation

These are Material UI components.

Component	Purpose
Container	Main wrapper
Paper	Elevated surface
Typography	Text display
TextField	Input field
Button	Button
Grid	Responsive layout
MenuItem	Dropdown item
Box	Generic wrapper
Divider	Horizontal line
Card	Card UI
InputAdornment	Prefix/Suffix inside inputs

---

import axios from "axios";

Explanation

Axios is used for API requests.

Used later for:

axios.post(...)

Icon Imports

import CheckCircleRoundedIcon from "@mui/icons-material/CheckCircleRounded";

Purpose

Success icon after form submission.

---

import { Zoom } from "@mui/material";

Purpose

Animation component.

Used for success card animation.

---

import RestartAltRoundedIcon from "@mui/icons-material/RestartAltRounded";

Purpose

Reset/restart icon.

---

2. Initial Form Values

const INITIAL_LEAD_VALUES = {

Explanation

Defines default form state.

This object is used for:

- initial values
- reset values

Core Fields

leadName: "",
inputType: "",
leadOwner: "",

Meaning

Initial empty values.

Customer Fields

customerName: "",
customerAddress: "",
customerEmail: "",
customerPhone: "",
enquiryDate: "",
Customer details.

Classification Fields

defenceOrNonDefence: "",
businessDomain: "",
businessMarket: "",
Business classification data.

Misc Fields

requirement: "",
remarks: "",
Extra information.

Why Outside Component?

This lives outside the component to prevent re-creation on every render

Important Concept

If declared inside component:

- object recreated every render
- performance issue

Outside component:

- created once only

---

3. BQ Stages Array

export const BQ_STAGES = [

Explanation
Stores Budgetary Quotation workflow stages.

"Scope of Work Study",
"Feasibility Study",
Pipeline stages.

Why Exported?
export const
Means other files can import this array.

---

4. Common Field Styles
export const commonFieldStyles = {
Purpose
Reusable styles for all TextFields.
Avoids repeated code.

Input Label Styles
inputLabelProps: {
Styles label text.

---

fontSize: "1.2rem",
fontWeight: 600,
Makes labels larger and bold.

Shrink Label Styling
"&.Mui-shrink": {
Targets floating label state.

---

fontSize: "0.9rem",
Smaller floating label.

TextField Styles
textFieldSx: {
Reusable field styling.

Outlined Input Root
"& .MuiOutlinedInput-root": {
Styles main input box.

Hover Effect

"&:hover": { boxShadow: "0 0 10px #bbdefb" },
Blue glow on hover.

Focus Effect
"&.Mui-focused": {
Styles active field.

---

boxShadow: "0 0 15px #90caf9",
Blue glow when focused.

Legend Styling
"& .MuiOutlinedInput-notchedOutline legend": {
Controls outline gap for floating label.

Label Positioning
"& .MuiInputLabel-root.Mui-shrink": {
Fixes floating label positioning.

Currency Adornment
inputProps: {
Adds prefix to input.

---

startAdornment: <InputAdornment position="start">₹</InputAdornment>,
Shows ₹ symbol inside field.

---

5. Multiline Props
const multilineProps = { multiline: true, rows: 2 };
Purpose
Reusable textarea settings.
Equivalent to:
multiline
rows={2}

---

6. STATUS_CHIP Object
const STATUS_CHIP = {
Purpose
Stores color configuration for statuses.

Example:
Draft: { bg: "#f0f3f8", color: "#5a6880" },
Draft status colors.

⚠️ Currently not used in file.

---

7. API Endpoint

const LEAD_ENQUIRY_API = "/lead-enquiry";

Purpose

Stores backend API route.

---

8. User from LocalStorage

let user = JSON.parse(localStorage.getItem("user")) || {};

Explanation

Gets logged-in user data.

If no user:

{}

used as fallback.

---

9. Main Component

const LeadEnquiryForm = (props) => {

Creates component.

Props Destructuring

const { serverIp, setValue, changeSubmitValue } = props;

Meaning

Prop	Purpose
serverIp	Backend URL
setValue	Change parent tab
changeSubmitValue	Notify parent submission

---

10. Submitted Data State

const [submittedData, setSubmittedData] = useState(null);

Purpose

Stores API response after submission.

---

11. Dynamic Dropdown State

const [dynamicOptions, setDynamicOptions] = useState({

Stores dropdown values dynamically.

Example:

inputType: ["EOI", "RFI", "RFE", "Custom Input"],

Dropdown options.

---

12. Common Card Styles

const commonCardStyles = {

Reusable card styling.

---

backdropFilter: "blur(10px)",

Glassmorphism effect.

---

boxShadow: "0 6px 20px rgba(0,0,0,0.06)",

Soft shadow.

---

13. React Hook Form Setup

```
const {
  control,
  handleSubmit,
  reset,
  formState: { errors },
} = useForm({
```

Explanation

Variable	Purpose
control	Connects fields
handleSubmit	Handles form submit
reset	Clears form
errors	Validation errors

Default Values

defaultValues: INITIAL_LEAD_VALUES

Initial form data.

---

14. onSubmit Function

const onSubmit = (data) => {

Runs when form submitted.

Formatting Data

const formattedData = {

Creates backend payload.

Mapping Fields

leadName: data.leadName,

Copies form values.

Operator Details

operatorId: user.id || "E-291536",

Uses logged-in user ID.

Fallback if missing.

Console Log

console.log(

"Frontend Lead Enquiry Data : ",

Debugging output.

---

15. API Request

axios

.post(serverIp + LEAD_ENQUIRY_API, formattedData)

Explanation

Sends POST request.

Example:

http://localhost:5000/lead-enquiry

Success Response

.then((response) => {

Runs when API succeeds.

Store Submitted Data

setSubmittedData(response.data.data);

Stores backend response.

Notify Parent

changeSubmitValue(true);

Tells parent component submission succeeded.

Reset Form

reset();

Clears fields.

Scroll to Top

window.scrollTo({ top: 0, behavior: "smooth" });

Smooth scroll to success card.

Error Handling

.catch((error) => console.log(error.message));

Logs API errors.

---

16. Reset Handler
const handleReset = () => {
Manual reset button logic.

---

reset();
Clears form.

---

setSubmittedData(null);
Removes success screen.

---

17. Download JSON Handler
const handleDownloadJSON = () => {
Allows downloading response as JSON.

Convert to JSON String
const dataStr = JSON.stringify(submittedData, null, 2);
Pretty JSON formatting.

Create Blob
const dataBlob = new Blob([dataStr], { type: "application/json" });
Creates downloadable file object.

Create URL
const url = URL.createObjectURL(dataBlob);
Temporary browser URL.

Create Anchor
const link = document.createElement("a");
Creates invisible download link.

Set Download Name
link.download = `domestic-leads-${Date.now()}.json`;
Dynamic filename.

Trigger Download
link.click();
Starts download.

Cleanup

URL.revokeObjectURL(url);
Removes temporary memory URL.

---

18. JSX Return
return (
UI rendering starts.

Main Container
<Container maxWidth="lg" sx={{mb:4}}>
Page wrapper.

Paper Component
<Paper
Creates elevated background.

Glassmorphism Styling
background: "rgba(255,255,255,0.9)",
backdropFilter: "blur(14px)",
Modern glass UI effect.

Conditional Rendering
{submittedData ? (
Meaning
If form submitted:
→ show success screen
Else:
→ show form

---

19. Success Screen
<Zoom in={submittedData !== null ? true : false}>
Zoom animation.

Success Card
<Card
Displays success content.

Success Icon
<CheckCircleRoundedIcon
Green success tick.

Success Message

Form Submitted Successfully!
Displayed after API success.

Reference Number
{submittedData.leadReferenceNo}
Shows generated lead reference.

Submit Another Button
onClick={() => {
Resets screen.

---

changeSubmitValue(false);
Updates parent state.

View Submitted Data Button
onClick={() => setValue(1)}
Changes parent tab to View Data.

---

20. Form Section
<form onSubmit={handleSubmit(onSubmit)}>
React Hook Form wrapper.

Section Cards
Each section uses:
<Card sx={commonCardStyles.cardLayoutSx.sx}>
Reusable card styling.

Example Field
<Controller
 name="leadName"
Connects field to form system.

Controller Purpose
Controller is needed because MUI TextField is controlled component.

Render Function
render={({ field }) => (
Provides field props automatically.

TextField
<TextField

{...field}
Injects:

- value
- onChange
- onBlur

Validation Errors
error={!!errors.leadName}
Shows red border if error exists.

Helper Text
helperText={errors.leadName?.message}
Displays validation message.

Dropdown Field
select
Converts TextField into dropdown.

Dropdown Options
{dynamicOptions.inputType.map((dt) => (
Loops through dropdown values.

Menu Item
<MenuItem key={dt} value={dt}>
Creates dropdown option.

Email Validation
pattern: {
Regex validation.

Email Regex
/^[A-Z0-9._%+-]+@[A-Z0-9.-]+\.[A-Z]{2,}$/i
Checks valid email format.

Phone Validation
value: /^[0-9+]*$/,
Allows only numbers and +.

Date Field
type="date"
Browser date picker.

Shrink Label

shrink: true,

Important for date inputs.

Multiline Field

{...multilineProps}

Adds textarea behavior.

Submit Button

type="submit"

Triggers form submission.

Hover Animation

transform: "scale(1.05)",

Button enlarges slightly.

Reset Button

onClick={handleReset}

Clears form manually.

Component Export

export default LeadEnquiryForm;

Allows import in other files.

Overall Flow of This Component

User Opens Form

↓

Fill Fields

↓

React Hook Form Tracks Data

↓

Validation Happens

↓

Submit Button Click

↓

onSubmit()

↓

axios POST API Call

↓

Backend Response

↓

Success Card Displayed

↓

User Can:

- Submit Another
- View Submitted Data

Major Concepts Used

Concept	Used
React Hooks	✅
useState	✅
React Hook Form	✅
Controller	✅
Form Validation	✅
Axios API Calls	✅
Conditional Rendering	✅
Dynamic Dropdowns	✅
Material UI	✅
Responsive Grid	✅
Animations	✅
Glassmorphism UI	✅

Important Architecture Understanding

This component acts as:

Role	Description
Form Manager	Handles inputs
Validation Engine	Validates data
API Client	Sends backend requests
UI Renderer	Displays sections
State Manager	Tracks success/data
Parent Communicator	Sends updates to parent

File Reference:

Here is the **detailed line-by-line explanation** of your TabLeadEnquiry.js file.
This component is mainly acting as a **Main Parent Component / Dashboard Tab Manager** for the Lead Enquiry module.
File Citation:

---

1. Import Statements

```
import React, { useEffect, useState, useMemo } from "react";
```

Explanation

This line imports:

Import	Purpose
React	Required to create React components
useEffect	Used for side effects like API calls
useState	Used for storing component state
useMemo	Used for memoization (performance optimization)

```
import { Container, Typography, Box, Divider, Tabs, Tab } from "@mui/material";
```

Explanation

These are UI components from **Material UI (MUI)**.

Component	Purpose
Container	Wraps the page content
Typography	Displays text/headings
Box	Generic layout wrapper
Divider	Horizontal separator line
Tabs	Creates tab navigation
Tab	Individual tab item

```
import axios from "axios";
```

Explanation

Axios is used for API calls.

⚠️ In this file it is imported but NOT USED.
You can remove it safely unless future use is planned.

---

2. Import Icons

```
import AddCircleOutlineRoundedIcon from "@mui/icons-material/AddCircleOutlineRounded";
import VisibilityOutlinedIcon from "@mui/icons-material/VisibilityOutlined";
import CloudUploadOutlinedIcon from "@mui/icons-material/CloudUploadOutlined";
```

Explanation

These are Material UI icons.

Icon	Used For
AddCircleOutlineRoundedIcon	Create Data tab
VisibilityOutlinedIcon	View Data tab
CloudUploadOutlinedIcon	Bulk Upload tab

---

3. Import Custom Components

import FormLeadEnquiry from "./FormLeadEnquiry";
import ViewLeadEnquiry from "./ViewLeadEnquiry";
import ExcelBulkUpload from "./ExcelBulkUpload";

Explanation

These are your own child components.

Component	Purpose
FormLeadEnquiry	Form for creating lead
ViewLeadEnquiry	Displays lead records
ExcelBulkUpload	Upload leads using Excel

---

import { getServerIp } from "../../utils/getServerIp";

Explanation

Imports utility function that fetches backend/server IP.
Used later inside useEffect.

---

4. Dropdown Constants

const DROP_DOWN_LISTS = {

Explanation

Creates a constant object containing all dropdown values.
This acts like a centralized master data storage.

Example:
inputType: ["EOI", "RFI", "RFE", "RFQ", "RFP", "Custom Input"],

Meaning

Dropdown options for Input Type field.

---

leadOwner: ["Umesha A", "Solomon", "Praveen"],

Meaning

Possible lead owners.

---

defenceAndNonDefence: ["Non-Defence", "Defence"],

Meaning

Dropdown for project type.

All remaining arrays are similar dropdown options.
This structure is good because:

- Reusable
- Easy to manage
- Centralized configuration

---

5. Styles Object

const styles = {

Explanation

This object contains all custom styles used in component.
Acts like CSS-in-JS.

Container Styles

container: {
 mt: 0,
 py: 1,
 borderRadius: 4,
},

Meaning

Property	Meaning
mt	margin-top
py	padding top-bottom
borderRadius	rounded corners

Title Styles

title: {
 fontWeight: 600,

Meaning

Semi-bold heading.

---

fontSize: { xs: "1.5rem", sm: "1.75rem", md: "2.125rem" },

Responsive Font Sizes

Screen Size	
xs	Mobile
sm	Tablet
md	Desktop

---

background: "linear-gradient(45deg, #0d47a1, #42a5f5, #1e88e5)",

Meaning

Creates gradient color effect.

WebkitBackgroundClip: "text",

WebkitTextFillColor: "transparent",

Meaning

Makes gradient appear inside text.

Very modern UI effect.

Divider Styles

divider: {

Styles horizontal line.

Tabs Styles

tabs: {

Customizes MUI Tabs.

"& .MuiTabs-flexContainer": {

Targets internal MUI class.

---

justifyContent: { xs: "flex-start", md: "center" },

Meaning

- Mobile → start aligned
- Desktop → center aligned

"& .MuiTab-root": {

Styles each tab button.

---

textTransform: "none",

Prevents uppercase conversion.

"& .Mui-selected": {

color: "#0d47a1 !important",

},

Selected tab color.

"& .MuiTabs-indicator": {

Styles the underline indicator.

---

6. Main Component

const LeadEnquiryTab = () => {

Explanation

Creates functional component.

This is the main exported page.

---

7. State Variables

const [tabValue, setTabValue] = useState(0);

Purpose

Stores active tab index.

Value	Tab
0	Create Data
1	View Data
2	Bulk Upload

---

const [submitSuccess, setSubmitSuccess] = useState(false);

Purpose

Tracks whether form submitted successfully.

Used to refresh data.

---

const [serverIp, setServerIp] = useState("");

Purpose

Stores backend server IP.

---

8. Memoized User Data

const user = useMemo(

Purpose

Caches user data.

Prevents repeated localStorage parsing.

() => JSON.parse(localStorage.getItem("user")) || {},

Explanation

Gets logged-in user from browser localStorage.

[]

Meaning

Runs only once.

---

9. Tab Change Handler

const handleTabChange = (event, newValue) => {

Purpose

Runs when user clicks a tab.

---

setTabValue(newValue);

Meaning

Updates active tab.

---

10. Form Submit Handler

const handleFormSubmitSuccess = (status) => {

Purpose

Receives success status from child component.

---

setSubmitSuccess(status);

Updates state.

---

11. useEffect Hook

useEffect(() => {

Purpose

Runs side effects.

Internal Async Function

const fetchIp = async () => {

Async function to fetch server IP.

---

const nodeserverIp = await getServerIp();

Waits for server IP.

---

setServerIp(nodeserverIp);

Stores IP in state.

Error Handling

```
catch (err) {
  console.error("Error setting server IP:", err);
}
```

Logs errors.

Function Call

fetchIp();

Actually executes async function.

Dependency Array

}, [submitSuccess]);

Meaning
Runs again whenever submitSuccess changes.
Useful for refreshing data after submission.

---

12. Render Helper Function
const renderTabContent = () => {
Purpose
Returns different component based on active tab.

Case 0 → Create Data
case 0:
Returns form component.

<FormLeadEnquiry
Child component.

---

serverIp={serverIp}
Passes server IP.

---

setValue={setTabValue}
Allows child to change tab.

---

changeSubmitValue={handleFormSubmitSuccess}
Allows child to update submission status.

Case 1 → View Data
<ViewLeadEnquiry
Displays data table.

---

dynamicOptions={DROP_DOWN_LISTS}
Passes dropdown values.

Case 2 → Bulk Upload
<ExcelBulkUpload user={user} serverIp={serverIp} />
Passes user + server IP.

Default Case
default:
return null;
If no tab matched.

---

13. JSX Return Section

return (

UI rendering starts.

Main Container

<Container maxWidth="xl" sx={styles.container}>

Creates page wrapper.

Header Section

<Box sx={{ textAlign: "center", mb: 2 }}>

Centers heading.

<Typography variant="h4" sx={styles.title}>

Displays title.

Lead Enquiry

Heading text.

Divider

<Divider sx={styles.divider} />

Gradient horizontal line.

Tabs Section

<Tabs

Main navigation tabs.

---

value={tabValue}

Controls active tab.

---

onChange={handleTabChange}

Handles click events.

---

variant="scrollable"

Makes tabs scrollable on small screens.

---

scrollButtons="auto"

Auto show scroll buttons.

---

allowScrollButtonsMobile

Allows scroll buttons on mobile.

Tab 1

<Tab

icon={<AddCircleOutlineRoundedIcon />}

Create Data icon.

---

label="Create Data"

Tab label.

Tab 2

icon={<VisibilityOutlinedIcon />}

label="View Data"

View records tab.

Tab 3

icon={<CloudUploadOutlinedIcon />}

label="Bulk Upload"

Bulk upload tab.

Content Section

<Box sx={{ mt: 0 }}>{renderTabContent()}</Box>

Displays selected tab component dynamically.

Component Closing

);

};

Ends component.

Export Statement

export default LeadEnquiryTab;

Purpose

Makes component available for import in other files.

Overall Flow of This File

LeadEnquiryTab

│

├── Tabs Navigation

│ ├── Create Data

│ ├── View Data

│ └── Bulk Upload

│

├── Fetches Server IP

├— Gets Logged User
├— Handles Tab Switching
├— Passes Props to Child Components
└— Dynamically Renders Selected Component

Architecture Understanding

This file acts as:

Role	Description
Parent Component	Controls all child components
State Manager	Maintains active tab & shared data
Router-like UI	Changes content without page reload
Shared Data Provider	Passes props to children

Important React Concepts Used

Concept	Used?
Functional Component	✅
Hooks	✅
useState	✅
useEffect	✅
useMemo	✅
Props Passing	✅
Conditional Rendering	✅
Dynamic Rendering	✅
Material UI	✅

One Improvement Suggestion

Remove unused import:

import axios from "axios";

Currently unnecessary.

File Reference: